

CS486 SOLUTION - TEST 3

Based on the FINAL REVIEW structure; total: 10 points

Question 1 - ER Models

[1 pt]

Solution. Flight has key FNo. FlightLeg is identified by owning Flight plus partial key LegNo, giving PK (FNo,LegNo). Each Leg has exactly one Orig and one Dest Airport, modeled as two role-named N:1 relationships to Airport(Code). LegInstance is identified by FlightLeg plus FDate, giving PK (FNo,LegNo,FDate), and has an N:1 relationship to Aircraft(AircraftID).

Relational mappings: `Leg(FNo,LegNo,Orig,Dest,...)` with PK(FNo,LegNo), FK FNo to Flight, Orig/Dest to Airport. `Instance(FNo,LegNo,FDate,AircraftID,...)` with PK(FNo,LegNo,FDate), FK(FNo,LegNo) to Leg and AircraftID to Aircraft.

Question 2 - Query Languages

[2 pt]

Solution. (a)

$$\pi_{FNo}(\sigma_{Orig='SGN'}Leg) \cap \pi_{FNo}(\sigma_{Dest='HAN'}Leg).$$

(b)

$$\{a \mid Airport(a) \wedge \neg \exists l (Leg(l) \wedge l.Dest = a.Code)\}.$$

(c)

```
WITH d AS (
  SELECT FNo,FDate,SUM(SeatsBooked) AS total_booked
  FROM Instance GROUP BY FNo,FDate
), r AS (
  SELECT *, DENSE_RANK() OVER(PARTITION BY FNo ORDER BY total_booked DESC) rk
  FROM d
)
SELECT FNo,FDate,total_booked FROM r WHERE rk=1;
```

Question 3 - Database Constraints

[1 pt]

Solution. Row-local: CHECK (Orig <> Dest) on Leg.

Capacity rule:

$$\neg \exists i \exists a (Instance(i) \wedge Aircraft(a) \wedge i.AircraftID = a.AircraftID \wedge i.SeatsBooked > a.Capacity).$$

An INSERT/UPDATE trigger on Instance joins inserted rows to Aircraft and rejects any row with SeatsBooked > Capacity. A standard FK only proves that the aircraft exists; it cannot compare SeatsBooked to a value in another table.

Question 4 - Serialization & Concurrency

[2 pt]

Solution. Edges: A gives $T_1 \rightarrow T_3$ and $T_2 \rightarrow T_3$; B gives $T_1 \rightarrow T_2$; C gives $T_2 \rightarrow T_1$ (from $w_2(C)$ before $w_1(C)$) and $T_3 \rightarrow T_1$ (from $r_3(C)$ before $w_1(C)$). Thus $T_1 \rightarrow T_2 \rightarrow T_1$ is a cycle; not conflict-serializable.

Reads of A by T1/T2 occur before T3's write, so no dirty read there. $r_2(B)$ follows $w_1(B)$, so T2 reads from T1; commit order has c_1 before c_2 , satisfying recoverability for that dependency. $r_3(C)$ follows $w_2(C)$, so T3 reads from T2, but c_3 occurs before c_2 in the stated order c_1, c_3, c_2 ; therefore not recoverable. Also not cascadeless because dirty reads occur.

2PL: growing phase acquires locks and releases none; shrinking phase releases locks and acquires no new ones. Acquiring any new lock after the first unlock switches back to growing and violates 2PL.

Question 5 - Recovery

[1 pt]

Solution. Winner T1 committed; loser T2 did not. Under UNDO-only, the database may contain loser updates, so undo T2 in reverse order: restore $C := 70$, then $B := 60$. T1's A update is not redone because the protocol requires committed effects to have been forced to stable storage before commit; hence committed transactions need no REDO.

Question 6 - Functional Dependencies & Normalization

[1 pt]

Solution. $B^+ = \{B, D\}$ (no rule continues from D alone). A^+ : A gives B,C; B gives D; CD gives E; E gives G; G gives A, so A^+ is all attributes. Thus A is a candidate key. G is also a candidate key because $G \rightarrow A$ and then A reaches all. E is also a key: $E \rightarrow G \rightarrow A \rightarrow BC$, then B gives D. C alone is not.

A BCNF violation is $B \rightarrow D$ because B is not a superkey. A lossless BCNF decomposition step is $R_1(B, D)$ and $R_2(A, B, C, E, G)$. Since $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BD = R_1$, the step is lossless.

Question 7 - Concurrency Control Execution

[1 pt]

Solution. Interleaving: T1 reads $X=5, Y=5$ and decides true; before T1 writes, T2 also reads $X=5, Y=5$ and decides true; T1 writes $X=0$; T2 writes $Y=0$. Final (0,0). Each transaction individually checked invariant $X + Y \geq 10$, but together they violate the intended resource invariant. This is write skew, typically prevented by serializable isolation/predicate locking (not merely row-level locking on the row each transaction later writes).

Question 8 - Query Optimization

[1 pt]

Solution. One month is about $20,000/24 \approx 833$ data pages if dates are uniformly distributed and the B+ tree is clustered. Cost is about $3 + 833 = 836$ I/Os versus 20,000 for a full scan, so the clustered B+ tree is strongly preferred. The hash index on AircraftID does not support a date range predicate and is irrelevant unless the query also has an equality predicate on AircraftID.